# 3488. Closest Equal Element Queries

Solved

Medium | Topics | Companies | Hint

You are given a **circular** array `nums` and an array `queries`.

For each query `i`, you have to find the following:

- The **minimum** distance between the element at index `queries[i]` and **any** other index `j` in the **circular** array, where `nums[j] == nums[queries[i]]`. If no such index exists, the answer for that query should be -1.

Return an array `answer` of the **same** size as `queries`, where `answer[i]` represents the result for query `i`.

**Example 1:**

> **Input:** `nums = [1,3,1,4,1,3,2], queries = [0,3,5]`
>
> **Output:** `[2,-1,3]`
>
> **Explanation:**
>
> - Query 0: The element at `queries[0] = 0` is `nums[0] = 1`. The nearest index with the same value is 2, and the distance between them is 2.
> - Query 1: The element at `queries[1] = 3` is `nums[3] = 4`. No other index contains 4, so the result is -1.
> - Query 2: The element at `queries[2] = 5` is `nums[5] = 3`. The nearest index with the same value is 1, and the distance between them is 3 (following the circular path: `5 -> 6 -> 0 -> 1`).

**Example 2:**

> **Input:** `nums = [1,2,3,4], queries = [0,1,2,3]`
>
> **Output:** `[-1,-1,-1,-1]`
>
> **Explanation:**
>
> Each value in `nums` is unique, so no index shares the same value as the queried element. This results in -1 for all queries.

**Constraints:**

- `1 <= queries.length <= nums.length <= 10^5`
- `1 <= nums[i] <= 10^6`
- `0 <= queries[i] < nums.length`

# Python:

```
class Solution:
    def solveQueries(self, nums: List[int], queries: List[int]) -> List[int]:
        n = len(nums)
        mp = defaultdict(list)

        # store indices
        for i in range(n):
            mp[nums[i]].append(i)

        ans = []

        for q in queries:
            v = mp[nums[q]]

            # only one time present
            if len(v) == 1:
                ans.append(-1)
                continue

            pos = bisect_left(v, q)
            res = float('inf')

            # left neighbor
            left = v[(pos - 1) % len(v)]
```

```
            d1 = abs(q - left)
            res = min(res, min(d1, n - d1))

            # right neighbor
            right = v[(pos + 1) % len(v)]
            d2 = abs(q - right)
            res = min(res, min(d2, n - d2))

            ans.append(res)

        return ans
```

## JavaScript:

```
/**
 * @param {number[]} nums
 * @param {number[]} queries
 * @return {number[]}
 */
var solveQueries = function(nums, queries) {
    const n = nums.length;

    const positions = new Map();

    for (let i = 0; i < n; i++) {
        if (!positions.has(nums[i])) {
            positions.set(nums[i], []);
        }
        positions.get(nums[i]).push(i);
    }

    const answer = new Array(n).fill(-1);

    for (const pos of positions.values()) {
        const m = pos.length;

        if (m === 1) continue;

        for (let i = 0; i < m; i++) {
            const curr = pos[i];

            const prev = pos[(i - 1 + m) % m];
            const next = pos[(i + 1) % m];

            let distPrev = Math.abs(curr - prev);
```

```
            distPrev = Math.min(distPrev, n - distPrev);

            let distNext = Math.abs(curr - next);
            distNext = Math.min(distNext, n - distNext);

            answer[curr] = Math.min(distPrev, distNext);
        }
    }

    return queries.map(idx => answer[idx]);
};
```

# Java:

```
class Solution {
    public List<Integer> solveQueries(int[] nums, int[] queries) {
        int n = nums.length;
        Map<Integer, List<Integer>> map = new HashMap<>();

        // store indices
        for (int i = 0; i < n; i++) {
            map.computeIfAbsent(nums[i], k -> new ArrayList<>()).add(i);
        }

        List<Integer> ans = new ArrayList<>();

        for (int q : queries) {
            List<Integer> v = map.get(nums[q]);

            // only one time present
            if (v.size() == 1) {
                ans.add(-1);
                continue;
            }

            int pos = Collections.binarySearch(v, q);
            int res = Integer.MAX_VALUE;

            // left neighbor
            int left = v.get((pos - 1 + v.size()) % v.size());
            int d1 = Math.abs(q - left);
            res = Math.min(res, Math.min(d1, n - d1));

            // right neighbor
            int right = v.get((pos + 1) % v.size());
```

```
            int d2 = Math.abs(q - right);
            res = Math.min(res, Math.min(d2, n - d2));

            ans.add(res);
        }

        return ans;
    }
}
```